

Class Components vs Function Components in React

Modularity

Design self-contained, reusable components

Composability

Compose complex UIs from simple parts

Encapsulation

Keep internal state and implementation private

Interoperability

Ensure components communicate via contracts



INTRODUCTION

Understanding React Components

Props

Pass data into components

State

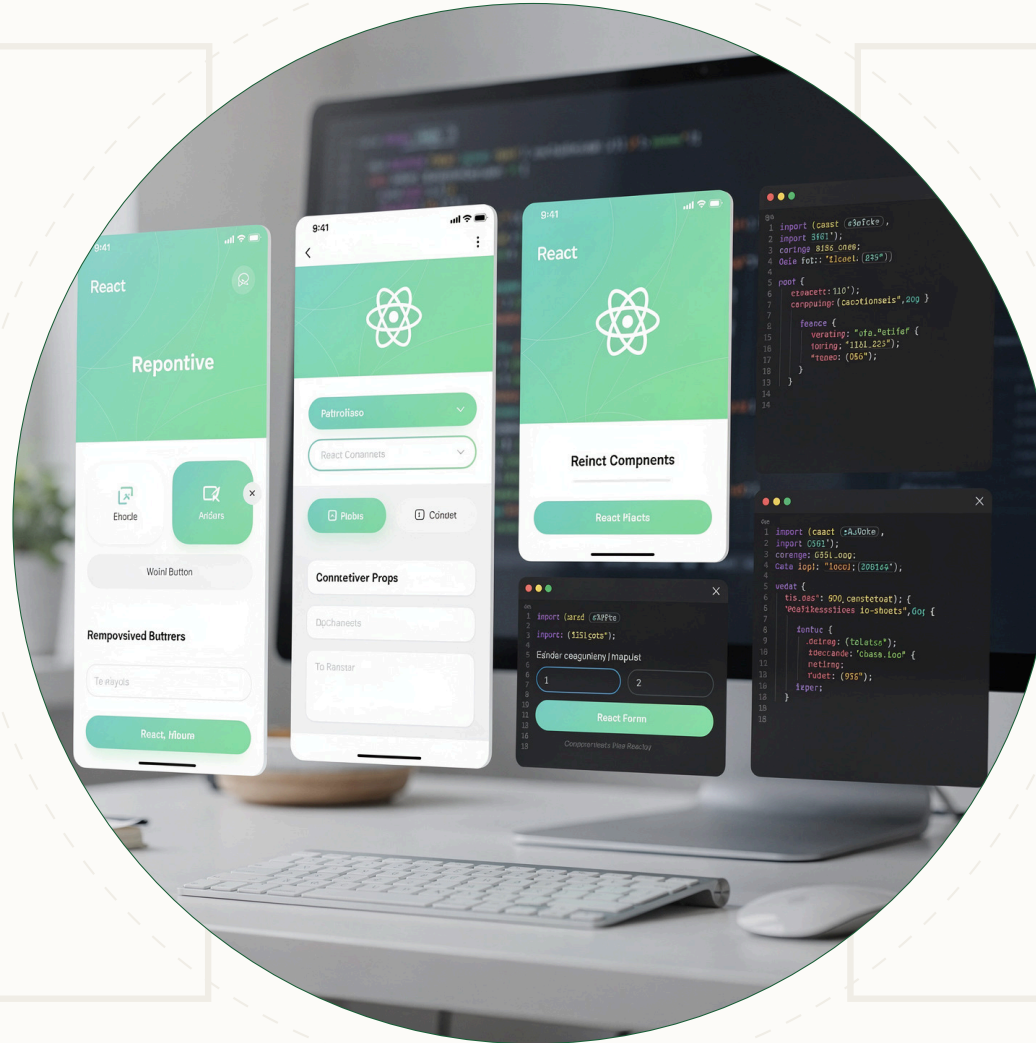
Manage local component data

Lifecycle

React to component events

Composition

Combine components together



React is a JavaScript library for building user interfaces through reusable, independent pieces called components. Each component encapsulates its own structure, logic, and styling, making applications modular and maintainable.

Components accept inputs called "props" and return React elements describing what should appear on the screen. React offers two primary approaches to creating components: Class Components and Function Components.

Core Characteristics

- Reusable building blocks
- Encapsulate UI logic
- Accept props as input
- Return renderable elements
- Two distinct implementation styles

CLASS COMPONENTS

Class Components: The Traditional Approach

Class Components are ES6 classes that extend from `React.Component`. They must include a `render()` method that returns JSX. These components have access to lifecycle methods and manage internal state through `this.state`.

Structure & Syntax

Class Components use the `class` keyword and extend `React.Component`. They require the `render()` method to return JSX elements.

State Management

State is initialised in the constructor using `this.state` and updated via `this.setState()`, which triggers re-rendering automatically.

Lifecycle Methods

Access to `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` for controlling behaviour at different stages.

```
class Welcome extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  componentDidMount() {
    console.log('Component mounted');
  }

  render() {
    return
```

Count: `{this.state.count}`

```
;}}
```



FUNCTION COMPONENTS

Function Components: The Modern Standard

Function Components are JavaScript functions that accept props and return JSX. Initially stateless, they gained full capabilities with React Hooks, allowing state management and lifecycle features without the complexity of classes.

Simple Syntax

Plain JavaScript functions that receive props as parameters and return JSX directly, without `render()` methods.

Hooks for State

The `useState` Hook enables state management, whilst `useEffect` replaces lifecycle methods with a unified approach.

Cleaner Code

No `this` keyword, no binding issues, and more concise syntax that improves readability and reduces boilerplate.

```
function Welcome(props) {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log('Component mounted or updated');
  }, []);

  return
```

Count: {count}

```
;}
```



COMPARISON

Key Differences Between Component Types

Aspect	Class Components	Function Components
Syntax	ES6 class extending React.Component	Plain JavaScript function
State	this.state and this.setState()	useState Hook
Lifecycle	componentDidMount, componentDidUpdate, etc.	useEffect Hook
this Keyword	Required for props, state, methods	Not needed
Code Length	More verbose, boilerplate code	Concise and readable
Performance	Slightly heavier	Lighter and faster
Learning Curve	Requires understanding of classes	Easier for beginners



PRACTICAL GUIDANCE

When to Use Each Component Type

Class Components

- Legacy codebases requiring maintenance
- Existing projects built before Hooks
- Working with older React libraries
- Error boundaries (not yet supported by Hooks)
- Team familiarity with object-oriented patterns

❑ Whilst still supported, Class Components are no longer the recommended approach for new React applications.

Function Components

- All new React projects and features
- Simpler component logic and state
- Better code reusability with custom Hooks
- Improved performance and bundle size
- Easier testing and maintenance

❑ React's official documentation now prioritises Function Components with Hooks as the standard approach.



SUMMARY

Key Takeaways & Learning Outcomes

01

Component Fundamentals

React components are reusable building blocks available in two forms: Class and Function, each with distinct syntax and capabilities.

03

Function Component Evolution

Modern standard using plain functions with Hooks (useState, useEffect) providing equivalent functionality with simpler syntax.

02

Class Component Characteristics

Traditional approach using ES6 classes, this keyword, lifecycle methods, and this.setState() for state management.

04

Modern Best Practices

Function Components with Hooks are now preferred for new development, offering better performance, readability, and maintainability.

Examination Focus Points

- Explain the structural differences between Class and Function Components
- Demonstrate state management in both component types
- Compare lifecycle methods with useEffect Hook
- Identify appropriate use cases for each component type
- Understand the role of Hooks in modern React development

